

# Leva.Framework.Core

---

`Leva.Framework.Core` is the shared vocabulary layer of ArchFramework. It defines stable contracts, IDs, result types, snapshots, synchronization helpers, and small value objects used by the rest of the framework.

Core exists so framework libraries can share the same language without creating dependency cycles. It contains concepts, not runtime execution, storage, UI, notifications, devices, providers, or application-specific behavior. `Leva.Framework.Core` depends only on .NET base libraries. Other framework libraries may depend on Core, but Core must not depend on Engine, Fakes, Storage, Presentation, Notifications, Authentication, provider implementations, or application projects.

## Project overview

---

Core is intentionally small and stable. A type belongs here only when it represents a framework-wide concept that many libraries can safely depend on. This includes contracts such as `IEvent`, `IState<TAccess>`, `IRoutine<TAccess>`, `IBehavior<TAccess>`, `ITransition`, and `ILogSink`; identifier values such as `StateId`, `RoutineId`, `EventId`, `RequestId`, and `AlarmId`; and shared values such as `Result`, `Error`, runtime entries, `Snapshot`, and small synchronization helpers.

Core does not decide how events are queued, how states are executed, how data is stored, or how UI is shown. Those responsibilities belong to Engine, provider libraries, or applications. Core only defines the shape of the concepts so those layers can communicate cleanly.

`Snapshot` is the durable recovery term used by the framework. It represents saved runtime/application state that can be persisted by a host or future storage provider.

## Files and classes

---

### Access, events, states, routines, and behaviors

`IAccess` - Base capability marker for typed access objects passed to states, routines, and behaviors. `IEvent` - Represents a typed event payload processed by the runtime and identified by an `EventId`. `IState<TAccess>` - Defines one application state with typed access, lifecycle handling, and event handling. `IRoutine<TAccess>` - Defines a reusable multi-step workflow that can process events while active. `IBehavior<TAccess>` - Defines reusable fallback behavior for events not handled by the current state or routine.

### Transitions, time, and logging

`ITransition` - Allows runtime objects to request state changes without directly controlling the state machine. `IClock` - Provides runtime time through an abstraction so framework logic can be deterministic and replaceable in tests. `SystemClock` - Production `IClock` implementation based on UTC system time. `ILogSink` - Receives diagnostic log entries emitted by the runtime or infrastructure.

The contract is write-only; memory and fake sinks may expose snapshots or clearing as concrete test conveniences.

## Identifier values

`AlarmId` - Identifies one alarm instance raised by the runtime or application. `RequestId` - Identifies a request that expects a correlated response or completion event. `EventId` - Identifies one event instance for logging, scheduling, cancellation, and correlation. `RoutineId` - Identifies a routine type or routine instance in runtime logs and bindings. `StateId` - Identifies an application state in transitions, snapshots, and runtime logs.

## Results and failures

`Error` - Represents a small structured failure value with code and message. `Result` - Represents success or structured failure without a returned value. `Result<T>` - Represents success or structured failure with a returned value.

## Runtime entries and values

`AlarmEntry` - Represents one active alarm or fault with principal, message, level, blocking flag, raised time, and optional properties. `AlarmLevel` - Defines alarm severity. `EventPriority` - Defines queue priority for events waiting in the engine queue. `RoutineStatus` - Describes routine lifecycle state. `Snapshot` - Captures durable runtime state that can be stored and later used to resume execution safely. `StatusEntry` - Represents one latest-known status value with source, name, value, updated time, and optional properties. `LogEntry` - Represents one diagnostic log item with source, message, category, level, created time, and optional properties. `LogLevel` - Defines diagnostic log severity.

## Synchronization helpers

`SyncList<T>` - Small synchronized list helper used by framework libraries that need safe snapshots of in-memory collections. `SyncDictionary<TKey, TValue>` - Small synchronized dictionary helper used by framework libraries that need safe snapshots of keyed in-memory collections.